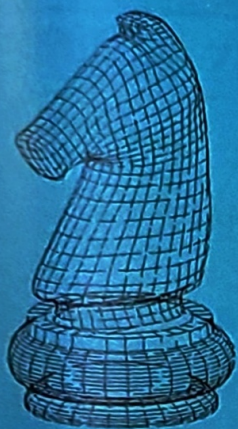


## 9장 가시면 결정



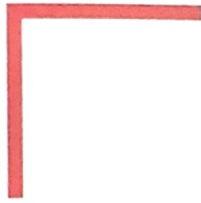
## 제 9 장

# 가시면 결정

### • 개관 •

3차원 그래픽스에서 장면 내의 객체를 구성하는 표면은 관찰자의 눈의 위치에 따라 보이기도 하고 다른 면에 가려 보이지 않기도 한다. 이러한 가시성을 올바르게 판단하여 화면에 디스플레이하는 것은 사실적인 그림을 만들어 내기 위해 반드시 필요하다. 가시면을 결정하는 알고리즘은 처리시간, 사용되는 기억용량, 장면의 특성 등 다양한 요인에 대해 더 우수한 성능을 낼 수 있는 다양한 것이 있다.

이 장에서는 장면 내의 보이는 면과 보이지 않는 면을 판별하기 위한 알고리즘의 원리와 대표적인 가시면 결정 알고리즘에 대해 소개한다.



---

#### 학습 목표

1. 가시면을 판별하는 기본적인 원리를 설명할 수 있다.
2. 그래픽스 시스템에서 다각형의 전면과 후면을 어떻게 정의할 수 있는지 설명할 수 있다.
3. 깊이 버퍼 알고리즘, 주사선 알고리즘, 깊이정렬 알고리즘 등 기본적인 가시면 결정 알고리즘을 설명할 수 있다.
4. OpenGL에서 제공하는 깊이 버퍼 관련 함수를 이용하여 은면을 제거함으로써 현실감 있는 그림을 만들 수 있다.

#### 주요 용어

가시면 결정 / 은면 제거 / 후면 제거 / 깊이 버퍼 알고리즘 / A-버퍼 알고리즘 / 주사선 알고리즘 / 깊이정렬 알고리즘 / 화가의 알고리즘

---



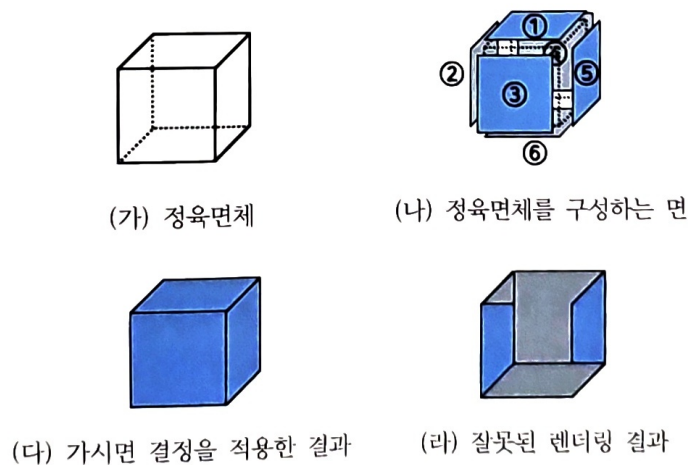
## 9.1 가시면 결정방법의 개념

### 9.1.1. 가시면 결정

어떠한 객체를 모델링하고 나면 이를 화면 위에 그려서 장면을 표현한다. 이처럼 주어진 모델로부터 그림을 만들어 내는 과정을 렌더링(rendering)이라고 한다. 이러한 렌더링 과정에는 뷰잉 과정에서 관찰자의 눈에 보이는 부분과 보이지 않는 부분을 구분하며, 표면에 텍스처를 입히거나 조명 및 음영 정보를 표현하는 처리 등이 포함될 수 있다.

특히 3차원 장면의 경우 장면을 구성하는 객체는 3차원 입체이며, 렌더링 과정을 통해 2차원 평면인 디스플레이 창에 이 객체를 투영하는 과정에서  $z$ 축 방향의 정보는 없어지게 된다. 그러므로 물체를 구성하는 평면을 그릴 때 주의하지 않으면 잘못된 그림이 만들어진다.

예를 들어, [그림 9-1]의 (나)와 같이 6개의 평면 조각으로 구성되는 정육면체를 화면에 그릴 때, 면의 위치를 고려하지 않고 ①~⑥번 면의 순서로 그릴 경우 (라)와 같이 보이지 않는 면이 앞쪽의 보이는 면을 가려 잘못된 결과를 만들어 낸다. 그러므로



[그림 9-1] 가시면과 은면



3차원 장면을 렌더링할 때는 어느 면이 관찰자의 눈에 보이는지 판별해야 한다. 컴퓨터 그래픽스에서 장면 내의 물체를 구성하는 표면 중에서 관찰자에게 보이는 면을 식별하는 기술을 가시면 결정(visible surface determination) 또는 역으로 은면 제거(hidden surface removal) 기법이라고 한다.

가시성을 판단하기 위해서는 3차원 정보가 남아 있어야 한다. 그러므로 뷰잉 파이프라인을 통해 정규화된 뷰 체적으로 변환 및 클리핑이 이루어진 후 뷰포트에 그리기 전에 가시성 결정이 이루어진다.

가시면 결정 알고리즘은 이미지공간 기법과 객체공간 기법으로 구분할 수 있다. 일반적으로 많이 사용되는 기법인 이미지공간 기법은 투영 평면의 각 픽셀 위치에 대해 점 대 점 단위로 가시성을 결정한다. 즉, 하나하나의 픽셀 좌표에 대해 이 점에서 보이는 표면이 무엇인지 검사하여 픽셀의 값을 결정하는 것이다. 이 방법은 모든 픽셀에 대해 이에 해당되는 가시면의 점을 결정해야 하므로 뷰포트의 크기에 의해 처리시간이 영향을 받는다. 또한 픽셀 단위로 가시성 결정을 위한 정보를 저장하는데, 그 결과 많은 양의 메모리가 필요하다. 깊이 버퍼 기법, A-버퍼 기법 등이 이미지공간 기법에 속하는 대표적인 방법이다.

객체공간 기법은 3차원 공간에 존재하는 객체의 공간적 순서에 따라 보이는 면(가시면)과 보이지 않는 면(은면)을 판별하는 방법이다. 특히 객체의 수가 적거나 이들이 위치하는 영역이 서로 겹치지 않고 흩어져 있을 경우에 객체공간 기법을 적용하면 효율적이다. 깊이정렬 알고리즘이 객체공간 기법에 해당되는 대표적인 방법이다.

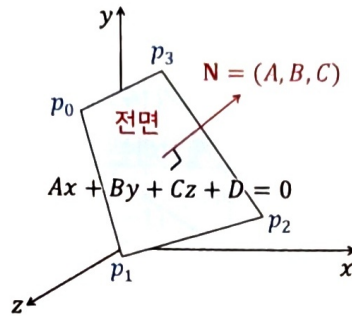
### 9.1.2. 후면 제거

3차원 공간에서 평면은 식 (9-1)에 의해 정의된다.

$$Ax + By + Cz + D = 0 \quad (9-1)$$

이때, 벡터  $(A, B, C)$ 는 이 평면에 수직인 법선 벡터  $\mathbf{N}$ 이다(그림 9-2 참조). 사실 평면은 2개의 면이 존재한다. 하나는 벡터  $\mathbf{N}$  방향을 향하는 면이고, 다른 하나는 그 반대 면이다. 평면을 화면에 디스플레이해야 하는 컴퓨터 그래픽스 응용에서는 이 두 면을 구분해야 하며,  $\mathbf{N}$  방향의 면을 전면,  $\mathbf{N}$ 의 반대 방향 면을 후면으로 정의한다.

사실 전면과 후면의 구분은 정의하기에 달려 있다. 벡터  $(-A, -B, -C)$  역시 식

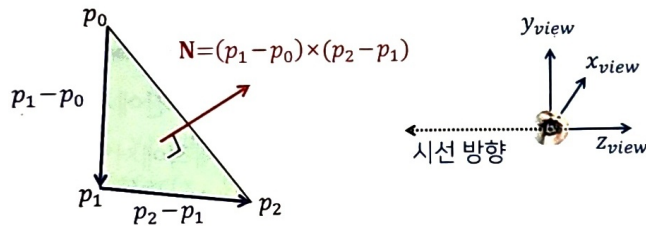


[그림 9-2] 평면의 전면 방향 및 법선 벡터

(9-1)의 평면에 대한 법선 벡터이기 때문이다. 따라서 어느 방향을 전면으로 할 것인지 결정하는 규칙이 필요하다. OpenGL에서는 다각형의 꼭짓점을 지정하는 순서에 따라 전면의 방향을 정의하는데, 시계 반대방향(ccw)을 디폴트로 정하고 있다.

블록다각형의 경우 정해진 순서에 따라 나열된 꼭짓점으로부터 법선 벡터를 쉽게 구할 수 있다. 만일 [그림 9-3]과 같이 CCW 방향으로  $p_0, p_1, p_2$ 를 나열하였다면 법선 벡터  $N$ 은 식 (9-2)와 같다.

$$N = (p_1 - p_0) \times (p_2 - p_1) \quad (9-2)$$



[그림 9-3] CCW 방향으로 꼭짓점이 나열된 블록다각형과 전면 법선 벡터

입체를 정의할 때는 입체의 바깥 면이 전면, 안쪽 면이 후면이 되도록 정의하는 것이 자연스럽다. [그림 9-1]과 같은 정육면체를 정의할 때도 이러한 규칙에 따라 정의한다. 그러므로 정육면체를 그림과 같이 경사평행투영할 경우 ①, ③, ⑤번 면은 전면이 관찰자를 향하고 있는 반면, ②, ④, ⑥번 면은 후면이 관찰자를 향하고 있다[그림 9-1 (나)]. 관찰자를 향하고 있는 전면은 다른 면에 의해 가려지지 않는다면 가시면에 해당

된다. 그러나 관찰자를 향하고 있는 후면은 눈에 보이지 않는 면이다.

일반적인 경우 후면의 수는 전체의 절반가량이다. 따라서 후면을 미리 제거한다면 가시면 결정 알고리즘이 검사해야 하는 면의 수가 줄어드는 효과가 있다. 장면 내의 표면 조각 중 후면을 렌더링 대상에서 제거하는 처리를 후면 제거(back-face culling, back-face removal)라고 한다.

만일 [그림 9-1]에서 후면 제거를 하지 않고 ①~⑥번의 순서에 따라 면을 그리면 (라)와 같은 그림을 얻게 된다. 그러나 후면 제거를 활성화한 후 동일하게 정육면체를 그리면 후면이 제거되어 (다)와 같은 정육면체를 그릴 수 있다.

관찰자의 시점에서 어떠한 면이 전면인지 후면인지는 법선 벡터가 시선 방향인지, 아니면 그 반대 방향인지에 의해 쉽게 구분된다. 제7장에서 살펴본 3차원 뷰잉 파이프라인에서 뷰잉 변환, 투영변환을 거쳐 정규화된 왼손 좌표계로 표현된 장면 내의 다각형은 법선 벡터의  $z$ 축 성분이 양이라면 후면이 관찰자에게 보일 것이다. 또는 식 (3-25)의 신발끈 공식으로 구한 면적의 부호를 기준으로 구분할 수도 있다. 만일 계산된 면적의 부호가 음이라면 후면이 관찰자에게 보이는 것이다.

---

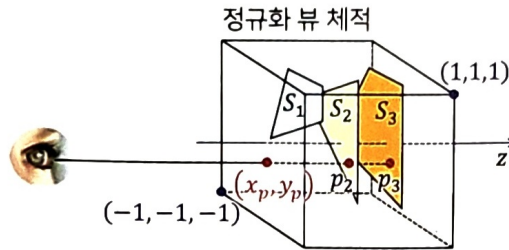
## 9.2 깊이 버퍼 알고리즘

---

깊이 버퍼 알고리즘(depth buffer algorithm)은 이미지공간에서 가시면을 결정하는 대표적인 알고리즘이다. 뷰잉 파이프라인의 정규화 뷰 체적에서 깊이는  $z$ 축 값으로 표현되므로  $z$ -버퍼 알고리즘이라고도 한다.

[그림 9-4]에서 정규화된 투영면상의 좌표  $(x_p, y_p)$ 는 뷰잉을 통해 계산된 정규화된 뷰 체적 내의 평면 조각  $S_2$ 의 점  $p_2$  및  $S_3$ 의 점  $p_3$ 가 투영되는 좌표이다. 이때,  $p_2$ 의 깊이, 즉  $z$ 좌표가  $p_3$ 의  $z$ 좌표보다 작으므로 점  $(x_p, y_p)$ 는  $p_2$ 에 의해 결정된다. 이와 같이 깊이 버퍼 알고리즘은 각 화면좌표에 투영되는 표면의 점 중 깊이가 가장 작은 것을 선택하여 화면에 그린다.

그런데 이러한 판단을 하기 위해 각 화면좌표마다 그 좌표에 투영되는 점들을 포함하는 모든 표면을 검사하여 찾아내는 것은 효율적이지 않다. 깊이 버퍼 알고리즘에서



[그림 9-4] 깊이 버퍼 알고리즘의 가시면 결정

는 래스터화한 그림을 저장하는 프레임 버퍼와 함께 깊이 버퍼(또는  $z$ -버퍼)를 사용한다. 정규화된 뷰 체적 내에 있는 표면 조각을 래스터화하여 프레임 버퍼에 픽셀값을 넣을 때, 우선 지금 계산된 픽셀의 깊이값과 현재 그 픽셀 위치의 깊이 버퍼의 값을 비교한다. 만일 현재 계산된 픽셀의 깊이가 깊이 버퍼에 있는 값보다 작다면 현재 픽셀이 관찰자에게 더 가까우므로, 이 값을 프레임 버퍼에 저장하면 된다. 이때, 깊이 버퍼의 값도 현재 픽셀의 깊이값으로 갱신한다. 만일 그렇지 않다면 지금 계산된 표면의 픽셀은 앞서 그린 다른 표면의 뒤에 위치하는 것이므로 은면에 해당되며, 따라서 그 픽셀은 프레임 버퍼에 넣지 않는다.

다음은 깊이 버퍼 알고리즘을 의사 코드로 작성한 것이다.

[소스코드 9-1] 깊이 버퍼 알고리즘

```

1  zBufferAlgorithm(Polygons)
2  {
3      for all (x, y) in Viewport {
4          FrameBuffer(x, y) = BACKGROUND_COLOR;
5          DepthBuffer(x, y) = 1.0;
6      }
7      for all p in Polygons {
8          for all (x, y) in p {
9              깊이 값 z 계산;
10             if (z < DepthBuffer(x, y)) {
11                 색상 값 P 계산;
12                 FrameBuffer(x, y) = P;
13                 DepthBuffer(x, y) = z;

```



|    |  |   |   |
|----|--|---|---|
| 14 |  |   | } |
| 15 |  |   | } |
| 16 |  | } |   |
| 17 |  | } |   |

알고리즘의 구현에서 다각형 내부 픽셀의  $z$ 좌표값은 점증적 방법으로 계산할 수 있다. 즉, 식 (9-1)의 방정식에 의해 정의되는 평면에서  $z$ 좌표는

$$z = \frac{-Ax - By - D}{C} \quad (9-3)$$

이다. 만일 픽셀 좌표  $(x, y)$  위치에서의  $z$ 좌표를 구하였다면  $(x+1, y)$  위치에서의  $z_{\text{next}}$ 는 식 (9-4)와 같이 구할 수 있다.

$$\begin{aligned} z_{\text{next}} &= \frac{-A(x+1) - By - D}{C} \\ &= z - \frac{A}{C} \end{aligned} \quad (9-4)$$

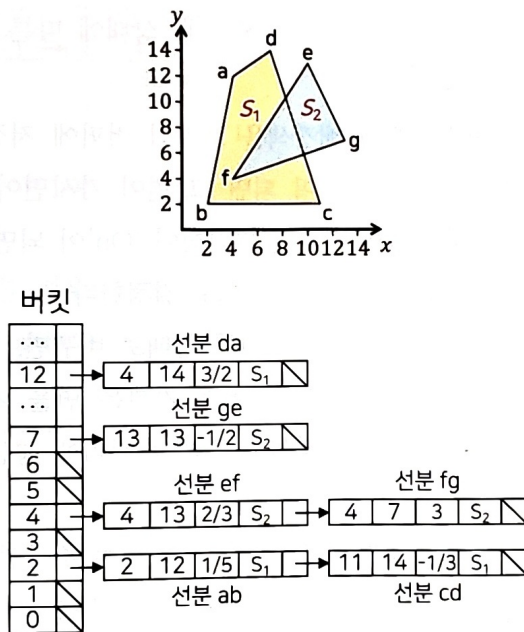
즉,  $x$ 좌표를 1 증가시킨 다음 점의  $z$ 좌표는 이전  $z$ 좌표에서 그 다각형에 대해 정해진 상수인  $A/C$ 를 뺀 값이다.

깊이 버퍼 알고리즘의 장점은 직관적이며 단순하다는 것이다. 각 표면은 임의의 순서로 래스터화하면 된다. 그러나 하나의 픽셀 위치에 대해 여러 번 계산하게 되는 단점이 있다. 특히 화면 해상도가 높아질수록 계산해야 할 픽셀 수도 증가하며, 그만큼 처리시간도 길어진다.

깊이 버퍼 알고리즘을 확장한 A-버퍼(Alpha-Buffer 또는 Anti-aliasing Buffer) 알고리즘이 있다. A-버퍼 알고리즘은 깊이 정보 외에 안티에일리어싱, 색혼합 등과 관련한 정보를 저장할 수 있도록 한다. A-버퍼의 각 픽셀 위치의 데이터는 깊이 필드와 표면 데이터 필드로 구성된다. 만일 어떤 픽셀에 하나의 표면만 투영되었을 때는 깊이 필드는 0보다 큰 깊이값을 가지게 되고, 표면 데이터 필드에는 색상, 투명도, 픽셀 점유도 등 해당 표면의 정보가 저장된다. 만일 어떤 픽셀에 여러 개의 표면이 투영되었을 때는 깊이 필드에는 음수가 저장되며, 이 경우 표면 데이터 필드에는 각 표면 정보를 저장하는 노드로 구성된 연결 리스트를 가리키는 포인터가 저장된다.

## 9.3 주사선 알고리즘

주사선 알고리즘은 행 단위로 가시성 검사를 수행하는 이미지공간 가시면 결정 알고리즘이다. 이 알고리즘에서는 우선 뷰 평면에 투영되는 모든 다각형 패싯(facet)을 구성하는 선분을 저장하는 선분 테이블을 만든다(이때, 수평 선분은 제외함). 각 선분은 2개의 끝점 중  $y$ 좌표값이 작은 끝점에 해당되는 선분 테이블 버킷에 저장된다. 한 버킷에 저장되는 선분들은 그 끝점의  $x$ 좌표 순서에 따라 정렬된다.



[그림 9-5] 주사선 알고리즘 처리를 위한 선분 테이블

[그림 9-5]는 투영면에 투영되는 두 다각형  $S_1$ 과  $S_2$ 의 변을 구성하는 선분들을 저장하는 선분 테이블을 보인 것이다. 선분 테이블에 저장되는 선분에 대한 항목에 저장되는 내용은 다음과 같다.

| $x_{y_{min}}$ | $y_{max}$ | $\Delta x$ | $S\_ID$ | ptr |
|---------------|-----------|------------|---------|-----|
|---------------|-----------|------------|---------|-----|

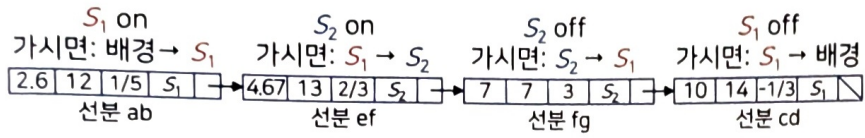
- $x_{y_{\min}}$ :  $y$ 좌표값이 작은 끝점의  $x$ 좌표
- $y_{\max}$ : 나머지 끝점의  $y$ 좌표
- $\Delta x$ : 다음 주사선으로 넘어갈 때 사용할  $x$ 증분(기울기의 역수)
- S\_ID: 선분이 속한 다각형의 ID
- ptr: 다음 선분 항목을 가리키는 포인터

가시면 결정을 하기 위한 검사는 다각형과 교차하기 시작하는 2번 주사선부터 시작한다. 매 주사선을 처리할 때마다 주사선과 교차하는 선분들의 목록인 활동 선분 목록을 갱신한 다음, 흘-짝 규칙에 따라 다각형의 내부 및 외부를 판별한다. 내부로 들어갈 경우 다각형의 상태는 'On'이 되고, 빠져나가면 'Off'가 된다. 이러한 상태의 변화가 발생할 때 가시면을 결정하기 위한 검사를 하는데, 상태에 따른 처리는 다음과 같다.

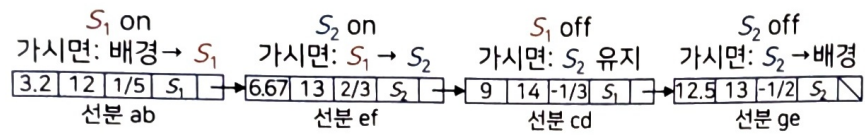
- ① 'On' 상태의 다각형이 없으면 배경색을 프레임 버퍼에 저장한다.
- ② 배경 상태에서 한 다각형이 'On'이 되면 그 면이 가시면이다.
- ③ 만일 한 다각형이 'On' 상태에서 다른 다각형이 'On'이 되면, 이들의  $z$ 좌표를 비교하여 관찰자에게 가까운 면을 가시면으로 결정한다.
- ④ 가시면이던 다각형이 'On' 상태에서 'Off' 상태로 바뀌면, 현재 'On' 상태인 다각형 중에서  $z$ 좌표를 비교하여 관찰자에게 가까운 면을 가시면으로 결정한다. 'On' 상태의 다각형이 없다면 배경색을 프레임 버퍼에 넣는다.

[그림 9-6]은 이러한 과정을 보여 준다. 예를 들어, 8번 주사선에 대한 처리를 하는 (나)의 경우 배경 영역에서 시작하여  $x = 3.2$ 에서 선분 ab를 만나  $S_1$ 이 'On'이 되며, 이제부터  $S_1$ 이 가시면이 된다. 다음  $x = 6.67$ 에서 선분 ef를 만나  $S_2$ 가 'On'이 되며, 현재 'On' 상태인  $S_1$ 과  $z$ 좌표를 비교한다.  $S_2$ 가 관찰자에게 가까우므로 이제부터  $S_2$ 가 가시면이 된다.  $x = 9$ 에서  $S_1$ 이 'Off' 상태로 바뀌지만, 가시면이 아니므로 계속하여  $S_2$ 의 색상으로 채워 나간다. 마지막으로  $x = 12.5$ 에서 선분 ge와 교차하여  $S_2$ 가 'Off'되며, 더 이상 'On'인 다각형이 없으므로 주사선의 나머지 픽셀은 배경색으로 채우면 된다.

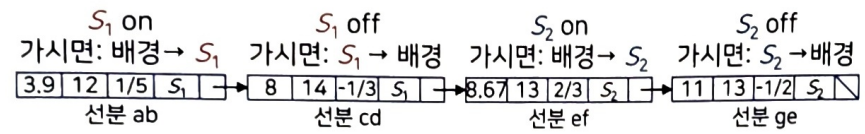
이러한 가시면 결정방법은 [그림 9-7]과 같이 다각형이 교차하는 경우 올바르게 처



(가) 5번 주사선



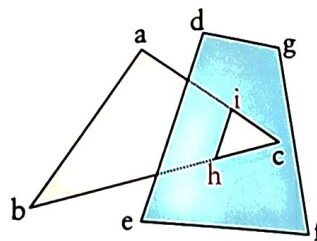
(나) 8번 주사선



(다) 11번 주사선

[그림 9-6] 활동 선분 목록과 가시면 결정

리되지 않는다. 이러한 경우는 하나의 다각형을 분할하여 처리해야 한다. 그림에서는 삼각형 abc를 다각형 abhi와 ihc로 나누어 처리하면 원하는 결과를 얻을 수 있다.



[그림 9-7] 교차하는 다각형의 분할



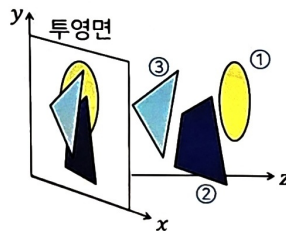
## 9.4 깊이정렬기법

깊이정렬기법은 일명 화가의 알고리즘(painter's algorithm)이라고도 한다. 이 이름은 [그림 9-8]에서와 같이 화가가 유화를 그릴 때 멀리 있는 풍경부터 그리고, 그 위에 가까이 있는 물체를 덧칠하여 그리는 것에서 착안한 것이다.



[그림 9-8] 화가의 알고리즘의 개념

깊이정렬기법에서는 먼저 깊이값의 내림차순으로 표면을 정렬한다. 그다음 [그림 9-9]와 같이 가장 큰 깊이값을 갖는 표면부터 차례로 래스터화한다. 그 결과, 깊이값이 작은 표면이 큰 깊이값을 갖는 표면을 그린 픽셀을 대체하게 되어 자연스럽게 가시면이 디스플레이된다. 이처럼 깊이정렬기법은 깊이에 따라 객체의 그리는 순서를 결정함으로써 가시면을 표현하는 객체공간 가시면 결정 알고리즘이다.

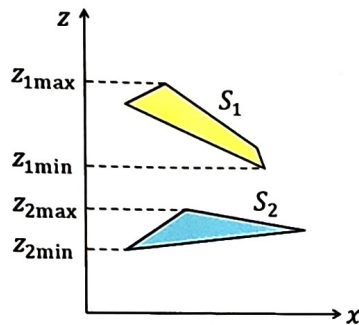


[그림 9-9] 깊이 순서에 따른 객체 그리기

3차원 공간에서 어떠한 다각형이 위치하는  $z$ 축 좌표는 하나의 값이 아니라 일정한 범위에 걸쳐 있다. 깊이정렬 알고리즘에서는 이 점을 고려하여 표면의 정렬을 해야 한

다. 깊이정렬 알고리즘의 처리과정은 다음과 같다.

먼저 각 다각형을 그 다각형의 가장 작은  $z$ 축 값의 내림차순에 따라 정렬하여 깊이 값이 가장 큰 다각형이 앞에 오도록 한다. 이 목록의 가장 앞에 있는 다각형이 이후의 다각형들과 깊이의 중첩이 있는지 검사한다. [그림 9-10]은 이해를 돕기 위해  $y=0$  평면에 투영해 놓은 두 평면이다.  $z_{\min}$ 의 값이 큰  $S_1$ 의  $z$ 축 범위가 이후에 있는 다각형  $S_2$ 의  $z$ 축 범위와 겹치는 부분이 없는 것을 볼 수 있다. 이처럼 목록의 선두에 있는 표면이 이후의 평면들과 깊이 중첩이 없다면 이 다각형  $S_1$ 을 래스터화하면 된다.



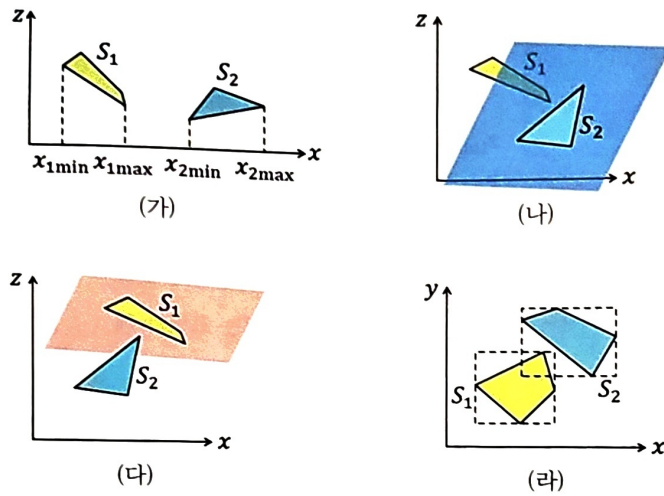
[그림 9-10] 깊이가 중첩되지 않는 표면의 예

만일 다각형  $S_1$ 이 이후에 있는 다각형과 깊이 중첩이 있다면 다음의 검사를 하며, 이 검사가 모두 거짓이면 두 다각형의 순서를 바꾸고, 어느 하나라도 참이면  $S_1$ 의 순서를 바꾸는 재정렬을 할 필요가 없다. 여기서  $S_1$ 을 포함하는 평면의 방정식은  $A_1x + B_1y + C_1z + D_1 = 0$ 이고,  $S_2$ 를 포함하는 평면의 방정식은  $A_2x + B_2y + C_2z + D_2 = 0$ 이라고 가정한다.

- ①  $x-y$  좌표상에서 두 다각형의 경계 직사각형이 서로 중첩하지 않는다[그림 9-11 (가)].
- ②  $S_1$ 이 중첩된 다각형  $S_2$ 를 포함하는 평면의 뒤에 위치한다[그림 9-11 (나)]. 이것은  $S_1$ 의 모든 꼭짓점  $(x_{s_1}, y_{s_1}, z_{s_1})$ 에 대해 부등식  $A_2x_{s_1} + B_2y_{s_1} + C_2z_{s_1} + D_2 < 0$ 을 만족하는지 검사하면 된다.
- ③ 중첩된 다각형  $S_2$ 가  $S_1$ 을 포함하는 평면의 앞에 위치한다[그림 9-11 (다)]. 이것은

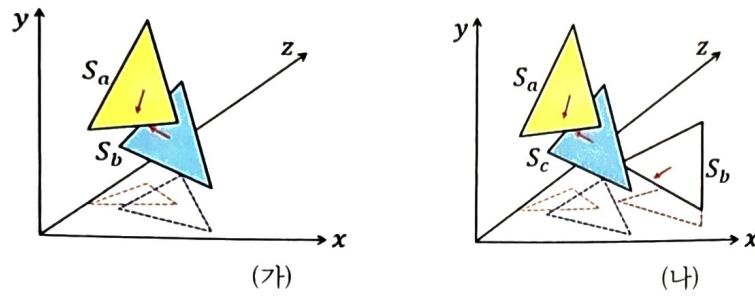
$S_2$ 의 모든 꼭짓점  $(x_{s_2}, y_{s_2}, z_{s_2})$ 에 대해 부등식  $A_1x_{s_2} + B_1y_{s_2} + C_1z_{s_2} + D_1 > 0$ 을 만족하는지 검사하면 된다.

- ④ 투영 평면에 투영된 두 평면이 중첩되지 않는다. [그림 9-11]의 (라)와 같이 투영된 두 다각형의 경계 직사각형이 중첩된다고 하더라도 실제 다각형은 중첩되지 않을 수 있다. 두 다각형의 선분 중 교차하는 것이 있는지, 교차하는 것이 없다면 한 다각형이 다른 다각형을 완전히 둘러싸고 있는지 검사(경계 직사각형의 크기가 작은 다각형의 임의의 꼭짓점이 큰 다각형의 내부에 있는지 판별하면 됨)해야 한다.



[그림 9-11] 재정렬을 하지 않아도 되는 조건

[그림 9-12]는 재정렬이 필요한 표면 배열의 예이다. (가)의 경우  $S_a$ 의 깊이가  $S_b$ 의 깊이보다 깊기 때문에  $(S_a, S_b)$ 의 순으로 나열될 것이다. 그렇지만 앞서 제시한 ①~④의 조건을 만족하지 않기 때문에 이들의 순서를  $(S_b, S_a)$ 로 바꾼다. 이제 ③의 조건을 만족하므로 이 순서대로 주사변환을 하면 된다. (나)의 경우 초기의 정렬된 목록은  $(S_a, S_b, S_c)$ 이다.  $S_a$ 와  $S_b$ 는 ①의 조건을 만족하므로 순서를 바꿀 필요가 없다. 그러나  $S_a$ 와  $S_c$ 는 ①~④의 조건을 만족하지 않기 때문에 이들의 순서를 바꾸어 목록은  $(S_c, S_b, S_a)$ 가 된다. 이제  $S_c$ 가  $S_b$ 나  $S_a$ 와 비교하여 ①~④ 중 만족하는 것이 있는지 판단한다. 이때,  $S_c$ 와  $S_b$ 는 이 조건을 모두 만족하지 않으므로 순서를 바꾼다. 그 결과



[그림 9-12] 재정렬이 필요한 표면 배열

목록은  $(S_b, S_c, S_a)$ 가 된다. 이제  $S_b$ 는  $S_c$ 와 조건 ②를 만족하고,  $S_a$ 와는 조건 ①을 만족하므로  $S_b$ 를 래스터화한다. 다음  $S_c$ 는  $S_a$ 와 조건 ③을 만족하므로  $S_c$ 를 래스터화하고, 이어서  $S_a$ 를 래스터화한다.

## 9.5 OpenGL의 가시면 결정 함수

### 9.5.1. OpenGL의 후면 제거

OpenGL에서는 다각형의 전면과 후면을 구분할 때 다각형을 정의하는 꼭짓점의 순서에 의해 정한다. 이 순서는 `glFrontFace` 함수에 의해 지정하는데, 디폴트는 시계 반대방향 순서이다.

```
void glFrontFace(GLenum mode);
```

- mode: 전면을 정의하는 꼭짓점 나열 순서
  - GL\_CCW: 시계 반대방향 순서 (디폴트)
  - GL\_CW: 시계 방향 순서

OpenGL에서는 다각형 선별기능을 이용하여 후면 또는 전면을 제거할 수 있다. 다



각형 선별기능은 다음과 같이 glEnable 및 glDisable 함수에 GL\_CULL\_FACE를 파라미터로 넣어 활성화하거나 해제한다.

```
glEnable(GL_CULL_FACE);
glDisable(GL_CULL_FACE);
```

제거할 면은 glCullFace 함수를 이용하여 지정한다.

```
void glCullFace(GLenum mode);
```

- mode: 제거할 면을 지정
  - GL\_BACK: 후면 제거(디폴트)
  - GL\_FRONT: 전면 제거
  - GL\_FRONT\_AND\_BACK: 전면 및 후면 제거(선, 점 등은 제외)

[소스코드 9-2]의 CullFace.cpp는 OpenGL의 다각형 선별함수를 이용하여 후면 제거를 하는 예이다.

#### [소스코드 9-2] CullFace.c

```
...   ※ 1~105행까지 [소스코드 7-2]와 동일함
106   void RenderCB( )
107   {
108       glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
109       glClear(GL_COLOR_BUFFER_BIT);    // 백색으로 화면 지움
110
111       TransMat4f viewMat;
112       Vec3f pOrg = { 150.0f, 120.0f, 100.0f };
113       Vec3f pRef = { 0.0f, 0.0f, 0.0f };
114       Vec3f vUp = { 0.0f, 1.0f, 0.0f };
115       viewingTrans(&viewMat, &pOrg, &pRef, &vUp);
116       TransMat4f perspectiveMat;
117       perspectivePrj(&perspectiveMat, -50.0f, 50.0f,
```

```

-50.0f, 50.0f, 70.0f, 350.0f);
118 TransMat4f tMat = perspectiveMat;
119 matmul_GTM(&tMat, &viewMat);
120 glUniformMatrix4fv(uGTransLocation, 1, GL_TRUE, tMat.mat);
121
122 glEnableVertexAttribArray(0);
123 glEnableVertexAttribArray(1);
124 glBindBuffer(GL_ARRAY_BUFFER, VBO[CUBE]);
125 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), 0);
126 glVertexAttribPointer(1, 4, GL_FLOAT, GL_FALSE, sizeof(Vertex),
    (const GLvoid*)sizeof(Vec3f));
127 glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);
128 glDrawArrays(GL_TRIANGLE_STRIP, 4, 4);
129 glDrawArrays(GL_TRIANGLE_STRIP, 8, 4);
130 glDrawArrays(GL_TRIANGLE_STRIP, 12, 4);
131 glDrawArrays(GL_TRIANGLE_STRIP, 16, 4);
132 glDrawArrays(GL_TRIANGLE_STRIP, 20, 4);
133 glDisableVertexAttribArray(0);
134 glDisableVertexAttribArray(1);
135
136 glFinish();
137 }
138
139 void InitVBOs()
140 {
141     Vertex Vertices[24] = { // 육면체의 꼭짓점 좌표 및 색
142         { -50.0f, -50.0f, -50.0f }, { 1.0f, 0.0f, 0.0f, 1.0f } },
143         { -50.0f, 50.0f, -50.0f }, { 1.0f, 0.0f, 0.0f, 1.0f } },
144         { 50.0f, -50.0f, -50.0f }, { 1.0f, 0.0f, 0.0f, 1.0f } },
145         { 50.0f, 50.0f, -50.0f }, { 1.0f, 0.0f, 0.0f, 1.0f } },
146
147         { -50.0f, -50.0f, 50.0f }, { 0.0f, 1.0f, 0.0f, 1.0f } },
148         { 50.0f, -50.0f, 50.0f }, { 0.0f, 1.0f, 0.0f, 1.0f } },
149         { -50.0f, 50.0f, 50.0f }, { 0.0f, 1.0f, 0.0f, 1.0f } },

```

```

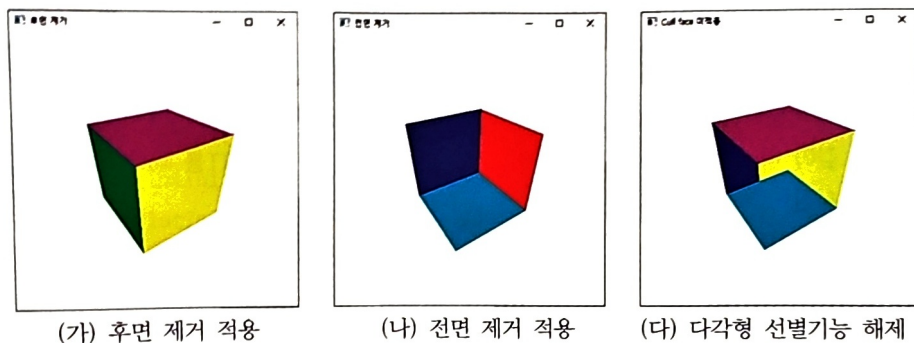
150         { {50.0f, 50.0f, 50.0f}, {0.0f, 1.0f, 0.0f, 1.0f} },
151
152         { {-50.0f, -50.0f, -50.0f}, {0.0f, 0.0f, 1.0f, 1.0f} },
153         { {-50.0f, -50.0f, 50.0f}, {0.0f, 0.0f, 1.0f, 1.0f} },
154         { {-50.0f, 50.0f, -50.0f}, {0.0f, 0.0f, 1.0f, 1.0f} },
155         { {-50.0f, 50.0f, 50.0f}, {0.0f, 0.0f, 1.0f, 1.0f} },
156
157         { {50.0f, -50.0f, -50.0f}, {1.0f, 1.0f, 0.0f, 1.0f} },
158         { {50.0f, 50.0f, -50.0f}, {1.0f, 1.0f, 0.0f, 1.0f} },
159         { {50.0f, -50.0f, 50.0f}, {1.0f, 1.0f, 0.0f, 1.0f} },
160         { {50.0f, 50.0f, 50.0f}, {1.0f, 1.0f, 0.0f, 1.0f} },
161
162         { {-50.0f, -50.0f, -50.0f}, {0.0f, 1.0f, 1.0f, 1.0f} },
163         { {50.0f, -50.0f, -50.0f}, {0.0f, 1.0f, 1.0f, 1.0f} },
164         { {-50.0f, -50.0f, 50.0f}, {0.0f, 1.0f, 1.0f, 1.0f} },
165         { {50.0f, -50.0f, 50.0f}, {0.0f, 1.0f, 1.0f, 1.0f} },
166
167         { {-50.0f, 50.0f, -50.0f}, {1.0f, 0.0f, 1.0f, 1.0f} },
168         { {-50.0f, 50.0f, 50.0f}, {1.0f, 0.0f, 1.0f, 1.0f} },
169         { {50.0f, 50.0f, -50.0f}, {1.0f, 0.0f, 1.0f, 1.0f} },
170         { {50.0f, 50.0f, 50.0f}, {1.0f, 0.0f, 1.0f, 1.0f} } };
171
172     glGenBuffers(N_VBOs, VBO);
173     glBindBuffer(GL_ARRAY_BUFFER, VBO[CUBE]);
174     glBufferData(GL_ARRAY_BUFFER, sizeof(Vertices), Vertices,
175                 GL_STATIC_DRAW);
176     glEnable(GL_CULL_FACE);
177     glCullFace(GL_BACK);
178 }
179
180 ※ 이하 main함수 [소스코드 7-2]와 동일함

```

InitVBOs 함수의 142~170행에서 정육면체의 각 면을 구성한 꼭짓점을 저장하는 데이터를 준비하였다. GL\_TRIANGLE\_STRIP으로 면을 그릴 수 있도록 꼭짓점을 순서대로 나열하였으며, 각 면마다 다른 색으로 그릴 수 있도록 서로 다른 색을 지정하였다. 꼭

꼭짓점 나열 순서를 특별히 지시하지 않았으므로 꼭짓점들을 시계 반대방향(CCW)으로 나열하여 전면을 정의하도록 한 것을 볼 수 있다. 175행에서는 다각형 선별기능을 활성화하였으며, 176행에서 후면을 제거하도록 지시하고 있다. `glCullFace` 함수에서 상수 `GL_BACK`은 디폴트이므로 이 문장은 생략해도 동일한 결과를 얻을 수 있다. 127~132행에서는 꼭짓점들을 4개씩 나누어 `GL_TRIANGLE_STRIP`으로 면을 그리도록 하였다.

[그림 9-13]의 (가)는 이 프로그램의 실행결과이다. 만일 `GL_BACK` 대신 `GL_FRONT`를 `glCullFace` 함수에 전달하면 전면이 제거되고, (나)와 같이 후면이 디스플레이된다. 또한 175행의 문장을 제거하면 다각형 선별기능이 해제되어 다각형을 그린 순서에 따라 화면에 그린다. 그 결과, (다)와 같이 의도한 형체가 올바르게 표현되지 않는다.



[그림 9-13] OpenGL의 다각형 선별기능

### 9.5.2. OpenGL의 깊이 버퍼 가시면 결정

깊이 버퍼를 이용하여 가시면 결정을 하기 위해서는 깊이 버퍼를 사용할 수 있도록 해야 한다. 이는 `glutInitDisplayMode` 함수에서 `GLUT_DEPTH` 비트를 통해 지정한다.

```
glutInitDisplayMode(GLUT_DEPTH | ... );
```

- ... 에는 그 외의 다른 모드 설정 비트가 나열됨

깊이 버퍼 가시면 결정 기능은 `glEnable` 함수에 `GL_DEPTH_TEST` 상수를 전달하여 활성화시킨다.



```
glEnable(GL_DEPTH_TEST);
```

깊이 버퍼는 `glClear` 함수를 통해 초기화한다. 이때, 채워 넣을 초기 깊이값을 `glClearDepth` 함수를 통해 0.0부터 1.0 범위의 값으로 지정할 수 있는데, 1.0이 아닌 다른 값을 지정함으로써 지정된 깊이 범위를 벗어나는 표면들을 렌더링 대상에서 제외할 수 있다. 이 값을 지정하지 않으면 깊이값은 일반적으로 사용되는 값인 1.0으로 초기화된다.

```
void glClearDepth(GLclampd depth);
```

- depth: 초기화에 사용될 깊이값으로 [0, 1] 범위로 제한됨

```
void glClear(GLbitfield mask);
```

- 깊이 버퍼 초기화를 지시하기 위해서는 `GL_DEPTH_BUFFER_BIT`를 mask 값에 포함시킴

예) `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);`

[소스코드 9-3]은 이러한 OpenGL 함수들을 활용하여 깊이 버퍼 가시면 결정을 적용하는 예이다.

#### [소스코드 9-3] CullFace.c

```
...   ※ 1~105행까지 [소스코드 9-1]과 동일함
106   void RenderCB( )
107   {
108       glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
109       glClearDepth(1.0);
110       // 화면과 깊이버퍼 지움
111       glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
112
113   ...   ※ 이하 [소스코드 9-1]의 RenderCB( ) 함수와 동일함
```

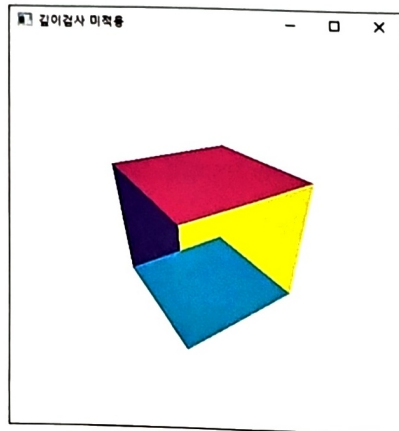
```

139 }
140
141 void InitVB0s( )
142 {
143     ...    ※ 이하 [소스코드 9-1]의 InitVB0s() 함수의 Vertices와 동일함
144
145     glGenBuffers(N_VB0s, VB0);
146     glBindBuffer(GL_ARRAY_BUFFER, VB0[CUBE]);
147     glBufferData(GL_ARRAY_BUFFER, sizeof(Vertices), Vertices,
148                 GL_STATIC_DRAW);
149     glEnable(GL_DEPTH_TEST);
150 }
151
152 int main(int argc, char** argv)
153 {
154     glutInit(&argc, argv);
155     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
156     glutInitWindowPosition(50, 100);
157     glutInitWindowSize(400, 400);
158     glutCreateWindow("깊이검사 적용");
159
160     ...    ※ 이하 main함수 [소스코드 9-1]과 동일함
161 }

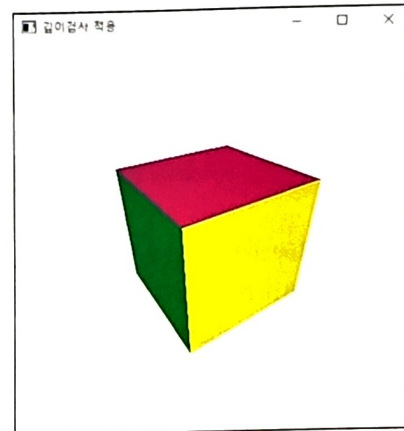
```

우선 183행에서 깊이 버퍼를 사용하도록 디스플레이 모드를 설정하고 있으며, 177행에서는 깊이 버퍼를 이용한 가시면 결정기능을 활성화한다. 렌더링 콜백 함수 RenderCB에서는 109행에서 깊이 버퍼를 지운 값으로 1.0을 설정한 후 111행에서 화면과 함께 깊이 버퍼를 지운 다음 정육면체를 그린다.

[그림 9-14]는 이 프로그램의 실행결과이다. (가)는 177행의 문장을 제거하여 깊이 버퍼 가시면 결정기능을 해제한 후 실행한 결과이다. 가시면 결정이 적용되지 않아 정육면체의 면을 그린 순서에 따라 겹쳐 그림으로써 의도와 다르게 표시되는 것을 볼 수 있다. 반면에, (나)는 177행의 문장으로 가시면 결정기능을 활성화하여 그린 결과이다.



(가) 깊이 검사 미적용



(나) 깊이 버퍼 알고리즘 적용

[그림 9-14] OpenGL의 깊이 버퍼 가시면 결정 활용 예

## 요약

- 가시면 결정이란 관찰자에게 보이는 표면을 식별하는 기술을 의미한다.
- 후면 제거는 후면이 관찰자를 향하고 있는 다각형을 렌더링 대상에서 제외함으로써 가시면 결정의 효율을 높이기 위한 방법이다.
- 깊이 버퍼 알고리즘은 투영 평면의 각 픽셀 위치에서 장면 내의 표면에 대한 깊이값을 비교하는 방법을 사용한다.
- A-버퍼 알고리즘은 각 픽셀당 깊이 정보 외에 색상, 투명도, 픽셀 점유도 등의 여러 가지 표면 데이터를 저장하는 버퍼를 사용한다.
- 주사선 알고리즘은 주사선 단위로 가시성 검사를 하며, 주사선이 다각형의 변과 교차하는 점에서 가시면을 결정한다.
- 깊이정렬기법은 깊이값의 내림차순으로 표면을 정렬한 후 가장 큰 깊이값을 갖는 표면부터 래스터화한다. 깊이 중첩이 있을 경우 재정렬이 필요할 수 있다.
- OpenGL의 다각형 렌더링 관련 함수

| 함수                                                                            | 내용                                                                        |
|-------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <code>glFrontFace(mode);</code>                                               | 다각형의 와인딩 순서 지정<br>mode: GL_CCW 또는 GL_CW                                   |
| <code>glEnable(GL_CULL_FACE);</code><br><code>glDisable(GL_CULL_FACE);</code> | 다각형 선별기능 활성화 및 해제                                                         |
| <code>glCullFace(mode);</code>                                                | 제거할 면을 선택<br>mode: GL_BACK(후면 제거)<br>GL_FRONT(전면 제거)<br>GL_FRONT_AND_BACK |
| <code>glutInitDisplayMode(GLUT_DEPTH  ...);</code>                            | 깊이 버퍼를 사용하도록 지정                                                           |
| <code>glEnable(GL_DEPTH_TEST);</code>                                         | 깊이 버퍼 가시면 결정기능 활성화                                                        |
| <code>glClearDepth(depth);</code>                                             | 깊이 버퍼를 초기화하는 값을 설정                                                        |
| <code>glClear(GL_DEPTH_BUFFER_BIT);</code>                                    | <code>glClearDepth</code> 함수로 지정한 값으로 깊이 버퍼를 초기화                          |





## 연습문제

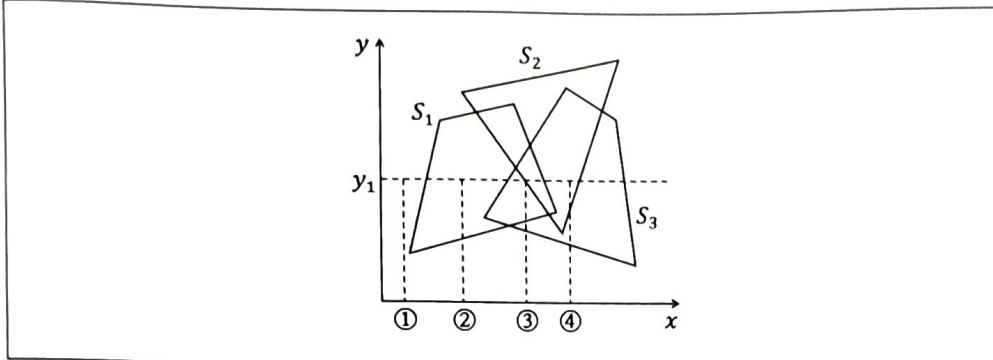
### 객관식

- 가시면 결정 알고리즘 중 이미지공간 기법에 대한 설명으로 올바른 것은?
  - ① 세계좌표상에 직접 적용된다.
  - ② 투영 평면의 각 픽셀 위치에 대해 점 대 점 단위로 가시성을 결정한다.
  - ③ 멀리 있는 물체를 먼저 그리고, 가까운 물체를 나중에 그린다.
  - ④ 화가의 알고리즘이라고도 한다.
- 어떠한 평면의 방정식이  $2x + 3y + z - 10 = 0$ 라 할 때, 다음 중 이 평면 전면의 법선 벡터는?
  - ① (1, 1, 1)
  - ② (1, 0, 0)
  - ③ (2, 3, 1)
  - ④ (1, 1, -5)
- 깊이 버퍼 가시면 결정 알고리즘에 대한 설명으로 올바른 것은?
  - ① 객체공간 가시면 결정 알고리즘이다.
  - ② 그래픽 메모리는 프레임 버퍼만 있으면 된다.
  - ③ 깊이값이 큰 다각형부터 작은 다각형 순서로 래스터화해야 한다.
  - ④ 어떠한 픽셀 위치에서 계산된 다각형의 깊이값이 기존 값보다 작다면 새로 계산된 색상값과 깊이값으로 기존 값을 대체한다.
- 다음 OpenGL 명령이 지시하는 것은?
 

`glEnable(GL_CULL_FACE);`

  - ① 다각형을 정해진 비트맵 패턴으로 채운다.
  - ② 다각형 선별기능을 활성화한다.
  - ③ 가시면 결정기능을 일시적으로 해제한다.
  - ④ 다각형의 전면 방향의 기준을 정의한다.

5. 주사선 알고리즘으로 가시면 결정을 하는 경우, 다음 그림의 주사선  $y_1$ 에서 다각형의 깊이값을 비교해야 하는 곳은? 단, 다각형이 서로 교차하지 않는다고 가정한다.



6. 다각형  $S_1$ 의 평면 방정식이  $A_1x + B_1y + C_1z + D_1 = 0$ 이고, 다각형  $S_2$ 의 평면 방정식이  $A_2x + B_2y + C_2z + D_2 = 0$ 이다. 다음 조건 중 깊이정렬기법에서  $z$ 축 좌표에 따라 정렬된 표면  $S_1$ 과  $S_2$ 의 순서를 바꾸지 않아도 되는 경우는? ( $S_1$ 이  $S_2$ 보다 정렬된 순서상 앞에 있다.)
- ①  $x-y$  좌표상에서  $S_1$ 과  $S_2$ 의 경계 직사각형이 서로 중첩되지 않는다.
  - ②  $S_1$ 이 중첩된 표면  $S_2$ 보다 앞에 위치한다.
  - ③  $S_1$ 의 모든 꼭짓점  $(x_1, y_1, z_1)$ 에서 부등식  $A_1x_1 + B_1y_1 + C_1z_1 + D_1 > 0$ 을 만족한다.
  - ④  $S_2$ 의 모든 꼭짓점  $(x_2, y_2, z_2)$ 에서 부등식  $A_1x_2 + B_1y_2 + C_1z_2 + D_1 < 0$ 을 만족한다.
7. OpenGL에서 깊이 버퍼 가시면 결정기능을 활성화시키는 명령은?
- ① `glEnable(GL_DEPTH_TEST);`
  - ② `glEnable(GL_POLYGON_OFFSET_FILL);`
  - ③ `glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);`
  - ④ `glFrontFace(GL_CW);`

정답 1. ② 2. ③ 3. ④ 4. ② 5. ③ 6. ① 7. ①

## 주관식

- 오른손 좌표계에서 CCW 방향으로 꼭짓점을 나열하여 다각형의 전면을 표현할 때, 세 꼭짓점  $(1, 0, 0)$ ,  $(0, 0, 1)$ ,  $(0, 1, 0)$ 의 순서에 따라 나열하였다면 이 다각형의 전면 법선 벡터는 무엇인가?
- $x-y$  평면과 평행하며 두 대각선 방향 꼭짓점이 각각  $(2, 2, 3)$ 과  $(6, 7, 3)$ 으로 정의된 빨강 직사각형  $S_1$ 과  $(4, 5, 5)$ 와  $(9, 8, 5)$ 로 지정된 파랑 직사각형  $S_2$ 가 있다.  $(0, 0, 20)$  위치에서  $(0, 0, 10)$  방향으로 바라보며, 뷰업 벡터가  $(0, 1, 0)$ 인 평행투영을 한다. 클리핑 원도는 세계좌표를 기준으로  $0 \leq x \leq 10$ ,  $0 \leq y \leq 10$ ,  $0 \leq z \leq 10$ 이다.  $100 \times 100$  크기의 뷰포트에  $S_1$ 과  $S_2$ 를 차례로 디스플레이하는 과정을 깊이 버퍼 알고리즘에 따라 수행하는 과정을 설명하라.
- 문제 2를 주사선 기법으로 처리하는 과정을 설명하라.
- 문제 2를 깊이정렬기법으로 처리하는 과정을 설명하라.
- 임의 크기의 다각형 2개를 세계좌표상에 배치하고, 깊이 버퍼 알고리즘에 따라 이를 화면에 디스플레이하는 OpenGL 프로그램을 작성하라.